

---

## Distributed Databases: Database Links and Transparency

---

In this Lab, we will explore concepts related to distributed databases (DDB) with a specific focus on database links. DDB refers to a database that is spread across multiple nodes, working together as a single logical database. In previous labs, we discovered data partitioning as a technique to manage large data volumes. Now, we will focus on how these partitioned databases can communicate and work together through database links.

Database links are essential components that allow a database to access objects stored in another database. They enable seamless querying, joining, and modifying of data across multiple database instances, creating a unified view of distributed data. This lab will demonstrate how to establish these communication pathways, combine distributed data through global views, and maintain data consistency through distributed transactions.

In this lab, you will:

- Set up database links between sites to enable cross-site queries
- Create global views and synonyms for transparent access to distributed data
- Implement and test distributed transactions across multiple sites

### ① Useful Notions & Syntaxes

#### (a) Database Links

Database links allow Oracle databases to access objects in other databases.

##### i) Creating a Database Link

```
1  -- Private database link (only accessible by the creator)
2  CREATE DATABASE LINK link_name
3  CONNECT TO username IDENTIFIED BY password
4  USING 'connect_string';
5
6  -- Query to view all database links owned by the current user
7  SELECT db_link, username, host
8  FROM user_db_links;
9
10 -- Public database link (accessible by all users)
11 CREATE PUBLIC DATABASE LINK link_name
12 CONNECT TO username IDENTIFIED BY password
13 USING 'connect_string';
```

##### ii) Using Database Links

```
1  -- Query a remote table
2  SELECT * FROM table_name@link_name;
3
```

```

4  -- Join a local table with a remote table
5  SELECT a.column1, b.column2
6  FROM local_table a
7  JOIN remote_table@link_name b ON a.key = b.key;
8
9  -- Insert into a remote table
10 INSERT INTO table_name@link_name VALUES (...);

```

## (b) Global Views and Synonyms

### i) Creating Global Views

```

1  -- Create a view that combines data from multiple sites
2  CREATE VIEW global_view_name AS
3  SELECT column1, column2, 'Site1' as site FROM table@site1_link
4  UNION ALL
5  SELECT column1, column2, 'Site2' as site FROM table@site2_link
6  UNION ALL
7  SELECT column1, column2, 'Site3' as site FROM table@site3_link;

```

### ii) Creating Synonyms

```

1  -- Create a synonym for easier access to a remote object
2  CREATE [PUBLIC] SYNONYM synonym_name FOR
   [owner.]object_name[@database_link_name];

```

## (c) Distributed Transactions

Oracle automatically handles distributed transactions using a two-phase commit protocol to ensure atomicity.

```

1  -- Example of a distributed transaction
2  BEGIN
3  -- Operation on local table
4  UPDATE local_table SET column1 = value1 WHERE condition;
5
6  -- Operation on remote table
7  UPDATE remote_table@site_link SET column1 = value2 WHERE
   condition;
8
9  -- The commit initiates a two-phase commit protocol
10 COMMIT;
11 END;
12 /

```

## ② Lab Demonstration: Distribution Strategy and Implementation Steps

This section provides a comprehensive demonstration of how to implement a distributed database system using database links. Follow these steps to understand and complete the lab.

### (a) Distribution Strategy Overview

The IMDB database has been horizontally partitioned across three separate database instances:

#### i) Site 1: Movies (*MOVIES\_SITE*)

- Contains all records where showType = 'movie'
- Related Staff, Principals, Episode and Rating data for movies is also stored here

#### ii) Site 2: TV Content (*TV\_SITE*)

- Contains all TV-related content, including: tvEpisode, tvSeries, tvMiniSeries, tvMovie, tvShort, and tvSpecial.
  - Contains all related TV staff, principal cast/crew, episode, and rating information
- iii) **Site 3: Other Content (OTHER\_SITE)**
- Contains remaining content types: short, video, and videoGame.
  - Contains all related data for these content types
- iv) **Global Access Layer**
- We'll implement a global administrator user that creates database links, views, and synonyms
  - This layer provides transparent access to all data across the three sites
  - End users will be able to query distributed data without needing to know which site contains which data

## (b) Implementation Steps

### i) Step 1: Setting Up Site Users

```

1  -- Connect as SYSDBA
2  sqlplus / as sysdba
3
4  -- Create users for each site
5  ALTER SESSION SET "_ORACLE_SCRIPT"=TRUE;
6
7  CREATE USER movies_site IDENTIFIED BY movies_pass QUOTA
   UNLIMITED ON USERS;
8  CREATE USER tv_site IDENTIFIED BY tv_pass QUOTA UNLIMITED ON
   USERS;
9  CREATE USER other_site IDENTIFIED BY other_pass QUOTA UNLIMITED
   ON USERS;
10 CREATE USER global_admin IDENTIFIED BY global_pass QUOTA
   UNLIMITED ON USERS;
11
12 -- Query to check if users were successfully created
13 SELECT username, created, user_id FROM all_users WHERE username
   IN ('MOVIES_SITE', 'TV_SITE', 'OTHER_SITE', 'GLOBAL_ADMIN');
14
15 -- Grant necessary privileges
16 GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE
   PROCEDURE, CREATE DATABASE LINK TO movies_site;
17 GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE
   PROCEDURE, CREATE DATABASE LINK TO tv_site;
18 GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE
   PROCEDURE, CREATE DATABASE LINK TO other_site;
19 GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE
   PROCEDURE, CREATE DATABASE LINK TO global_admin;
20
21 -- Grant select on IMDBMANAGER tables to site users
22 GRANT SELECT ON IMDBMANAGER.SHOW TO movies_site, tv_site,
   other_site, global_admin;
23 GRANT SELECT ON IMDBMANAGER.STAFF TO movies_site, tv_site,
   other_site, global_admin;
24 GRANT SELECT ON IMDBMANAGER.PRINCIPALS TO movies_site, tv_site,
   other_site, global_admin;
25 GRANT SELECT ON IMDBMANAGER.EPISODE TO movies_site, tv_site,
   other_site, global_admin;

```

```
26 GRANT SELECT ON IMDBMANAGER.RATING TO movies_site, tv_site,  
    other_site, global_admin;
```

## ii) Step 2: Creating and Populating Tables

Before setting up database links, we need to create and populate the tables at each site with data from IMDBMANAGER.

```
1  -- Connect to movies_site (Site 1)
2  CONNECT movies_site/movies_pass
3  -- Create the Show table
4  CREATE TABLE Show (
5      idShow INTEGER PRIMARY KEY,
6      showType VARCHAR2(50),
7      showTitle VARCHAR2(255),
8      startYear INTEGER,
9      endYear INTEGER
10 );
11 -- Populate with movie data from IMDBMANAGER
12 INSERT INTO Show
13 SELECT * FROM IMDBMANAGER.SHOW
14 WHERE showType = 'movie';
15 COMMIT;
16
17
18 -- Connect to tv_site (Site 2)
19 CONNECT tv_site/tv_pass
20 -- Create and populate the Show table
21 -- Create the Show table
22 CREATE TABLE Show (
23     idShow INTEGER PRIMARY KEY,
24     showType VARCHAR2(50),
25     showTitle VARCHAR2(255),
26     startYear INTEGER,
27     endYear INTEGER
28 );
29 -- Populate with TV data from IMDBMANAGER
30 INSERT INTO Show
31 SELECT * FROM IMDBMANAGER.SHOW
32 WHERE showType IN ('tvEpisode', 'tvSeries', 'tvMiniSeries',
33     'tvMovie', 'tvShort', 'tvSpecial');
34 COMMIT;
35
36
37 -- Connect to other_site (Site 3)
38 CONNECT other_site/other_pass
39 -- Create and populate the Show table
40 -- Create the Show table
41 CREATE TABLE Show (
42     idShow INTEGER PRIMARY KEY,
43     showType VARCHAR2(50),
44     showTitle VARCHAR2(255),
45     startYear INTEGER,
46     endYear INTEGER
47 );
48 -- Populate with other content data from IMDBMANAGER
49 INSERT INTO Show
```

```

49 SELECT * FROM IMDBMANAGER.SHOW
50 WHERE showType IN ('short', 'video', 'videoGame');
51 COMMIT;

```

### iii) Step 3: Creating Private Database Links Between Sites

Private database links are only accessible by the user who creates them. We'll create these links to allow each site to access the other sites.

```

1  -- Connect to the movies_site (Site 1)
2  CONNECT movies_site/movies_pass
3  -- Create links to other sites
4  CREATE DATABASE LINK tv_link
5  CONNECT TO tv_site IDENTIFIED BY tv_pass
6  USING 'localhost:1521/XE';
7  -- USING '(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=localhost)
8    (PORT=1521))(CONNECT_DATA=(SERVICE_NAME=ORCLCDB)))'
9  CREATE DATABASE LINK other_link
10 CONNECT TO other_site IDENTIFIED BY other_pass
11 USING 'localhost:1521/XE';
12
13 -- Connect to the tv_site (Site 2)
14 CONNECT tv_site/tv_pass
15 -- Create private links to other sites
16 CREATE DATABASE LINK movies_link
17 CONNECT TO movies_site IDENTIFIED BY movies_pass
18 USING 'localhost:1521/XE';
19 CREATE DATABASE LINK other_link
20 CONNECT TO other_site IDENTIFIED BY other_pass
21 USING 'localhost:1521/XE';
22
23 -- Connect to the other_site (Site 3)
24 CONNECT other_site/other_pass
25 -- Create private links to other sites
26 CREATE DATABASE LINK movies_link
27 CONNECT TO movies_site IDENTIFIED BY movies_pass
28 USING 'localhost:1521/XE';
29 CREATE DATABASE LINK tv_link
30 CONNECT TO tv_site IDENTIFIED BY tv_pass
31 USING 'localhost:1521/XE';
32
33
34 -- Connect to movies_site
35 CONNECT movies_site/movies_pass
36
37 -- Test query to TV_SITE using tv_link
38 SELECT COUNT(*) AS tv_show_count
39 FROM Show@tv_link;
40
41 -- View sample data from TV_SITE
42 SELECT idShow, showType, showTitle, startYear
43 FROM Show@tv_link
44 WHERE ROWNUM <= 5;

```

### iv) Step 4: Setting Up Public Database Links

Public database links are accessible by all users in the database. We'll create these to

allow global views that can be accessed by anyone.

```
1  -- Connect back as SYSDBA
2  CONNECT / AS SYSDBA
3  -- Create public database links
4  CREATE PUBLIC DATABASE LINK movies_public_link
5  CONNECT TO movies_site IDENTIFIED BY movies_pass
6  USING '(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=localhost)
7  (PORT=1521))(CONNECT_DATA=(SERVICE_NAME=ORCLCDB)))';
8
9  CREATE PUBLIC DATABASE LINK tv_public_link
10 CONNECT TO tv_site IDENTIFIED BY tv_pass
11 USING '(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=localhost)
12 (PORT=1521))(CONNECT_DATA=(SERVICE_NAME=ORCLCDB)))';
13
14 CREATE PUBLIC DATABASE LINK other_public_link
15 CONNECT TO other_site IDENTIFIED BY other_pass
16 USING '(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=localhost)
17 (PORT=1521))(CONNECT_DATA=(SERVICE_NAME=ORCLCDB)))';
```

The key differences between private and public database links are:

- Private links (Step 2) can only be used by the user who created them (e.g., only movies\_site can use the tv\_link it created)
- Public links (Step 3) can be used by any user in the database with appropriate privileges
- Private links are created with CREATE DATABASE LINK
- Public links are created with CREATE PUBLIC DATABASE LINK

#### v) Step 5: Creating Global Views for Transparent Access

```
1  -- Connect to global_admin
2  CONNECT global_admin/global_pass
3
4  -- Create global view for all shows
5  CREATE VIEW global_shows AS
6  SELECT idShow, showType, showTitle, startYear, endYear, 'Movies'
7  AS site
8  FROM Show@movies_public_link
9  UNION ALL
10 SELECT idShow, showType, showTitle, startYear, endYear, 'TV' AS
11 site
12 FROM Show@tv_public_link
13 UNION ALL
14 SELECT idShow, showType, showTitle, startYear, endYear, 'Other'
15 AS site
16 FROM Show@other_public_link;
```

Test the views :

```
1  -- Verify access to the global view
2  SELECT * FROM global_shows WHERE ROWNUM <= 10;
```

#### vi) Step 6: Create public synonyms

Synonyms provide a simpler way to access objects across schemas when working in a single database instance.

```
1  -- Connect as global_admin
2  CONNECT global_admin/global_pass
```

```

3
4 -- Create public synonyms for the global views
5 CREATE PUBLIC SYNONYM all_shows FOR global_admin.global_shows;
6
7 -- Create synonyms for direct access to remote tables
8 CREATE SYNONYM movies FOR Show@movies_public_link;
9 CREATE SYNONYM tv_shows FOR Show@tv_public_link;
10 CREATE SYNONYM other_content FOR Show@other_public_link;
11
12
13
14 -- Connect as global_admin
15 CONNECT global_admin/global_pass
16
17 -- Test the global synonym "all_shows" for the global view
18 SELECT * FROM all_shows
19 WHERE ROWNUM <= 5;
20
21
22 -- Test "tv_shows" synonym
23 SELECT * FROM tv_shows
24 WHERE showType = 'tvSeries' AND ROWNUM <= 3;

```

#### vii) Step 7: Implementing a Distributed Transaction

```

1 -- Connect as global_admin
2 CONNECT global_admin/global_pass
3
4 -- Begin a distributed transaction
5 BEGIN
6     -- Update movie titles from 2020 to uppercase in the movies
7     -- database
8     UPDATE Show@movies_public_link
9     SET showTitle = UPPER(showTitle)
10    WHERE startYear = 2020;
11
12    -- Update TV show titles from 2020 to uppercase in the TV
13    -- database
14    UPDATE Show@tv_public_link
15    SET showTitle = UPPER(showTitle)
16    WHERE startYear = 2020;
17
18    -- Commit the transaction (this will use two-phase commit)
19    COMMIT;
20 END;
21 /

```

#### viii) Step 8: Testing Distributed Queries

```

1 -- Connect to movies_site
2 CONNECT movies_site/movies_pass
3 -- Verify distribution of shows across sites
4 SELECT site, COUNT(*)
5 FROM all_shows
6 GROUP BY site;

```

## Queries

### (a) Basic Database Link Queries :

- i) Write a query to find the total number of shows by type across all sites using the public links.
- ii) Write a query to find all shows from the 1990s (startYear between 1990 and 1999) from both Movies and TV sites, but not from the Other site, using private database links. Connect as movies\_site.
- iii) Connect as tv\_site and find all shows that have the same title in both the TV site and Movies site.
- iv) Write a distributed transaction that connects as global\_admin and updates all show titles from the year 2020 to uppercase format in both the movies database and the TV database simultaneously.

## ④ Solutions for Lab Work Queries:

### i) Solution to Query 1: Total shows by site using public synonym

```

1  -- Connect to MOVIES_SITE
2  CONNECT movies_site/movies_pass
3
4  -- Query to find total number of shows by site using the public
   synonym
5  SELECT site, COUNT(*) AS total_shows
6  FROM all_shows
7  GROUP BY site
8  ORDER BY site;
```

This demonstrates that database links and synonyms provide location transparency, allowing users to access remote data seamlessly. Public synonyms enable any user to access global views without knowing the underlying complexity. A single query can aggregate data from multiple distributed sites, and the physical distribution of data is hidden from the end user.

### ii) Solution to Query 2: Total shows by type across all sites

```

1  -- Connect as any user with access to the public links
2  CONNECT global_admin/global_pass
3
4  -- Query to find total shows by type across all sites
5  SELECT showType, COUNT(*) AS type_count
6  FROM global_shows
7  GROUP BY showType
8  ORDER BY type_count DESC;
9
10 -- Alternative approach using UNION ALL
11 SELECT showType, COUNT(*) AS type_count
12 FROM (
13     SELECT showType FROM Show@movies_public_link
14     UNION ALL
15     SELECT showType FROM Show@tv_public_link
16     UNION ALL
17     SELECT showType FROM Show@other_public_link
18 )
```

```

19 GROUP BY showType
20 ORDER BY type_count DESC;

```

iii) **Solution to Query 3: Shows from 1990s in Movies and TV sites using private links**

```

1  -- Connect as movies_site
2  CONNECT movies_site/movies_pass
3
4  -- Find shows from the 1990s in both Movies and TV sites
5  SELECT 'Movies' AS site, idShow, showTitle, showType, startYear
6  FROM Show
7  WHERE startYear BETWEEN 1990 AND 1999
8  UNION ALL
9  SELECT 'TV' AS site, idShow, showTitle, showType, startYear
10 FROM Show@tv_link
11 WHERE startYear BETWEEN 1990 AND 1999
12 ORDER BY startYear, showTitle;

```

iv) **Solution to Query 4: Shows with same title in TV and Movies sites**

```

1  -- Connect as tv_site
2  CONNECT tv_site/tv_pass
3
4  -- Find shows with the same title in both TV and Movies sites
5  SELECT t.showTitle,
6         t.showType AS tv_show_type,
7         m.showType AS movie_show_type,
8         t.startYear AS tv_year,
9         m.startYear AS movie_year
10 FROM Show t
11 JOIN Show@movies_link m ON t.showTitle = m.showTitle
12 ORDER BY t.showTitle;

```

v) **Solution to Query 5: Distributed transaction to update 2020 show titles**

```

1  -- Connect as global_admin
2  CONNECT global_admin/global_pass
3
4  -- Begin a distributed transaction
5  BEGIN
6      -- Update movie titles from 2020 to uppercase in the movies
7      -- database
8      UPDATE Show@movies_public_link
9      SET showTitle = UPPER(showTitle)
10     WHERE startYear = 2020;
11
12     -- Update TV show titles from 2020 to uppercase in the TV
13     -- database
14     UPDATE Show@tv_public_link
15     SET showTitle = UPPER(showTitle)
16     WHERE startYear = 2020;
17
18     -- Commit the transaction (this will use two-phase commit)
19     COMMIT;
20 END;
/

```

```
21 -- Verify the updates were successful
22 SELECT 'Movies' AS site, idShow, showTitle, startYear
23 FROM Show@movies_public_link
24 WHERE startYear = 2020 AND ROWNUM <= 5
25 UNION ALL
26 SELECT 'TV' AS site, idShow, showTitle, startYear
27 FROM Show@tv_public_link
28 WHERE startYear = 2020 AND ROWNUM <= 5;
```

## 5 Homework

By the end of each Lab, a report should be done, that contains all the details of the elaborate work during the Lab session. The report is going to be submitted later on. **You can work in groups of three maximum.**